

Mid-term examination on Erlang

Christian Rinderknecht

25 April 2008

A *queue* is a like stack where items are pushed on one end and popped on the other end. Adding an item in a queue is called *enqueueing*, whereas removing one is called *dequeueing*. Compare the following figures:

Queue : ENQUEUE $\rightarrow$

a	b	c	d	e
---	---	---	---	---

 $\rightarrow$ DEQUEUE

Stack : PUSH, POP $\leftrightarrow$

a	b	c	d	e
---	---	---	---	---

Let **enqueue**(E,Q) be the queue where E is the last item in and Q is the remaining queue. Let **dequeue**(Q) be the pair {E,R} where E is the first item out of queue Q (if there is none, **dequeue**(Q) is undefined) and R is the remaining queue.

Question. Define **enqueue/2** and **dequeue/1** in the following two cases.

1. **A one-stack implementation.** A simple idea to implement one queue is to use one stack. In this case, ENQUEUE is simply PUSH and DEQUEUE removes the item at the bottom of the stack.
2. **A two-stack implementation.** We can implement one queue with two stacks instead of one: one for enqueueing, one for dequeueing.

ENQUEUE $\rightarrow$

a	b	c
---	---	---

d	e
---	---

 $\rightarrow$ DEQUEUE

So ENQUEUE is PUSH on the first stack and DEQUEUE is POP on the second. If the second stack is empty, we swap the stacks and reverse the (new) second:

If ENQUEUE $\rightarrow$

a	b	c
---	---	---

--

 $\rightarrow$ DEQUEUE ???

then ENQUEUE $\rightarrow$

--

a	b	c
---	---	---

 $\rightarrow$ DEQUEUE

Let the pair {S,T} denote the queue where S is the stack for enqueueing and T the stack for dequeueing.

Question. Which of the one-stack or two-stack implementation is the most efficient for dequeueing? What are the best and worst cases for each?